

1 Solution — GIAM 8.3

Problem 1

1.1 Problem

Determine a substitution rule — a consistent way of replacing one digit with another along the diagonal — so that a diagonalization proof showing the interval $(0, 1)$ is uncountable will work in *decimal*. Write up the proof.

1.2 Short Answer — The Rule and the Conclusion

Rule. For each $n \geq 1$, replace the diagonal digit $d_{n,n}$ by

$$e_n = \begin{cases} 2 & \text{if } d_{n,n} = 1, \\ 1 & \text{if } d_{n,n} \neq 1. \end{cases}$$

The constructed real $r = 0.e_1e_2e_3\cdots$ then has every digit in $\{1, 2\}$, so $r \in (0, 1)$, has a *unique* decimal representation, and differs from each listed r_n at position n — contradicting the assumption that all reals in $(0, 1)$ were listed. Hence $(0, 1)$ is uncountable.

1.3 The Hidden Hazard the Rule Has to Avoid

The decimal expansion of a real in $(0, 1)$ is not always unique: some reals have *two* representations. For example,

$$\begin{aligned} \frac{1}{2} &= 0.5000\cdots = 0.4999\cdots, \\ \frac{3}{4} &= 0.7500\cdots = 0.7499\cdots, \\ \frac{1}{10} &= 0.1000\cdots = 0.0999\cdots. \end{aligned}$$

The general rule (Section 8.3 of the textbook): a real $r \in (0, 1)$ has two decimal representations iff $r = k/10^m$ for some positive integers k, m (a “finite decimal”). In that case the two forms differ by ending in trailing 0’s vs. trailing 9’s.

The diagonal construction produces a *string* differing from each listed *string* at one position. But two different strings can represent the same number. So if our constructed string happens to be the *alternative* representation of some r_n on the list, the strings differ but the numbers don't — and the proof collapses.

Fix: pick the substitution rule so that the constructed string is guaranteed to be the unique representation of its number. That means: avoid both trailing 0's *and* trailing 9's in the constructed string.

1.4 The Rule and Why It Works

The rule " $e_n = 2$ if $d_{n,n} = 1$, else $e_n = 1$ " produces a string in which every digit is either 1 or 2. In particular:

- *No digit is 0.* So the constructed string is not the “trailing-9's” representation of any number (those representations require infinitely many 9's eventually, which the constructed string lacks — but the more direct issue is the trailing-0's rep, addressed next).
- *No digit is 9.* So the constructed string is not “all-9's” eventually, hence not a trailing-9's representation of a finite decimal.
- *Equivalently:* the constructed string is non-terminating (infinitely many non-zero digits) and not eventually-9's, so it is the *canonical* representation of its number, and that number has *no* alternative representation.

So the constructed real r has a unique decimal expansion — the constructed string itself. If r equalled some r_n , the listing's representation of r_n would have to *be* the constructed string, contradicting the construction's “differs at position n ” property.

1.5 The Proof Write-Up

Claim. The interval $(0, 1)$ is uncountable.

Proof (by contradiction). Suppose $(0, 1)$ is countable. Then there is a bijection $\mathbb{N} \rightarrow (0, 1)$, so the reals in $(0, 1)$ can be enumerated as

$$r_1, r_2, r_3, r_4, \dots$$

For each r_n , choose the *canonical* (non-trailing-9's) decimal expansion

$$r_n = 0.d_{n,1} d_{n,2} d_{n,3} d_{n,4} \cdots, \quad d_{n,k} \in \{0, 1, \dots, 9\}.$$

(For non-finite-decimal reals there is only one representation anyway; for finite decimals we choose the terminating one, padded with infinitely many trailing 0's.)

Apply the substitution rule along the diagonal:

$$e_n = \begin{cases} 2 & \text{if } d_{n,n} = 1, \\ 1 & \text{if } d_{n,n} \neq 1, \end{cases} \quad n = 1, 2, 3, \dots$$

Form the real

$$r = 0.e_1 e_2 e_3 e_4 \cdots.$$

We verify four things:

1. $r \in (0, 1)$. Each $e_n \in \{1, 2\}$, so $0.111 \cdots = 1/9 \leq r \leq 0.222 \cdots = 2/9$. In particular $r \in [1/9, 2/9] \subset (0, 1)$.

2. r has a unique decimal representation. A real $s \in (0, 1)$ has two decimal representations iff $s = k/10^m$ for some $k, m \in \mathbb{N}$, in which case the canonical representation terminates (ends in trailing 0's). The constructed string $0.e_1 e_2 \cdots$ has every digit in $\{1, 2\}$, so in particular *no digit is 0*. Hence the string is not the canonical representation of a finite decimal, so r is not a finite decimal, so r has a unique representation — and the constructed string $0.e_1 e_2 \cdots$ is that representation.

3. r differs from each r_n . Suppose $r = r_n$ for some n . The listed representation of r_n is $0.d_{n,1} d_{n,2} d_{n,3} \cdots$, which (since r has unique representation) must equal the constructed string $0.e_1 e_2 e_3 \cdots$ digit-by-digit. In particular $d_{n,n} = e_n$. But by the substitution rule $e_n \neq d_{n,n}$:

$$\begin{aligned} \text{if } d_{n,n} = 1 : & \quad e_n = 2 \neq 1 = d_{n,n}, \\ \text{if } d_{n,n} \neq 1 : & \quad e_n = 1 \neq d_{n,n}. \end{aligned}$$

So $e_n \neq d_{n,n}$, contradicting $d_{n,n} = e_n$.

4. *Contradiction*. So $r \in (0, 1)$ but r is not in the listing $r_1, r_2, \dots$, contradicting the assumption that the listing enumerates all of $(0, 1)$.

Hence $(0, 1)$ is not countable. ■

1.6 A Worked Example

To make the construction concrete, here is a small piece of a hypothetical listing with the rule applied:

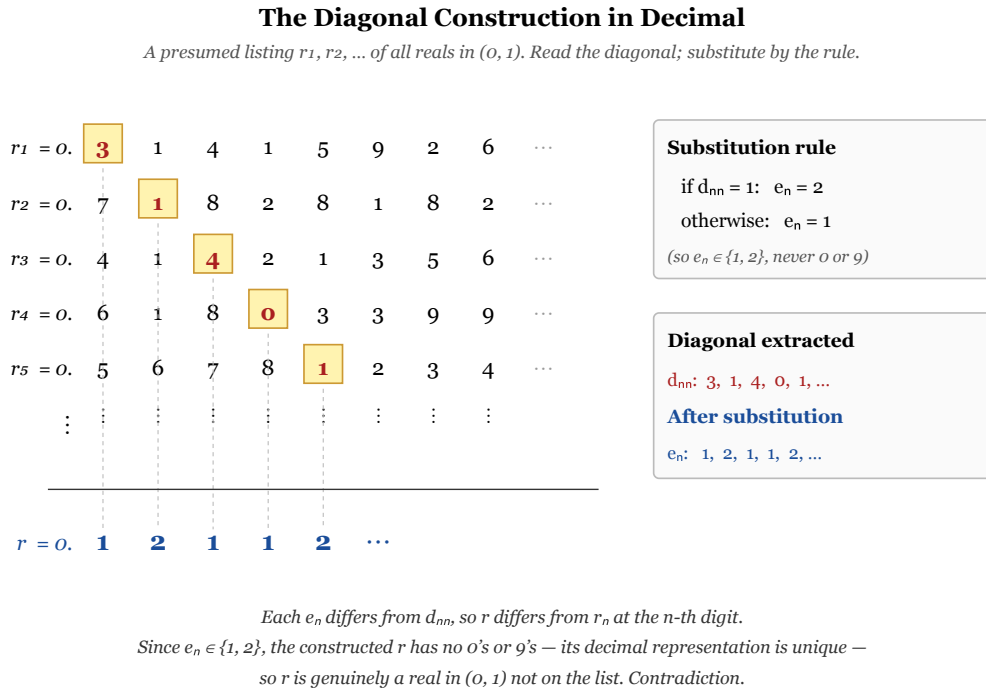


Figure 1.1: The diagonalization construction in decimal. The diagonal digits are 3, 1, 4, 0, 1, ...; applying the substitution rule $e_n = 1$ if $d_{n,n} \neq 1$, else 2 gives $e_n = 1, 2, 1, 1, 2, \dots$. The constructed $r = 0.12112\dots$ is in $(0, 1)$, has a unique decimal expansion (since no digit is 0 or 9), and differs from each r_n at position n .

Reading the diagonal digits **3, 1, 4, 0, 1, ...** and applying the rule gives $e_n = 1, 2, 1, 1, 2, \dots$, hence $r = 0.12112\dots$. By construction r differs from r_1 at position 1 (digit 1 vs 3), from r_2 at position 2 (2 vs 1), from r_3 at position 3 (1 vs 4), from r_4 at position 4 (1 vs 0), and from r_5 at position 5 (2 vs 1).

1.7 Remark — The Rule Is Not Unique

Any rule that produces $e_n \in \{1, 2, \dots, 8\}$ (avoiding 0 and 9) with $e_n \neq d_{n,n}$ works equally well — there are infinitely many valid choices, and the diagonal argument is robust under all of them. Alternatives that show up in different textbooks:

- *Switch 5 $\leftrightarrow$ 6*: $e_n = 6$ if $d_{n,n} = 5$, else $e_n = 5$.

- *Cycle*: $e_n = (d_{n,n} + 1)$ if $d_{n,n} \in \{1, 2, \dots, 7\}$, $e_n = 1$ if $d_{n,n} \in \{0, 8, 9\}$.
- “Always 5 unless that’s the diagonal, then 6”: $e_n = 5$ if $d_{n,n} \neq 5$, else $e_n = 6$.

The choice “ $e_n \in \{1, 2\}$ ” used above is simply the smallest convenient pair of safe digits.

What does *not* work: $e_n = (d_{n,n} + 1) \bmod 10$. This can produce $e_n = 0$ (when $d_{n,n} = 9$) or chains of 9’s, which lead exactly to the dual-representation hazard the rule is meant to avoid.

1.8 Remark — Why Decimal Is Easier Than Binary

The decimal argument is *easy* because the digit set $\{0, 1, \dots, 9\}$ is large enough to admit many “safe” digits — those avoiding both 0 and 9. Other bases have less room:

Safe Digits by Base (avoid 0 and b-1)			
<i>"Safe" = won't produce dual representations; need at least 2 to guarantee one differs from d_{nn}.</i>			
Base b	Digits	Safe digits ($\neq 0, \neq b-1$)	Status
2 (binary)	$\{0, 1\}$	$\{\}$ (empty)	fails (single digits)
3 (ternary)	$\{0, 1, 2\}$	$\{1\}$ (just one)	needs paired-digits or alt
4	$\{0, 1, 2, 3\}$	$\{1, 2\}$	works (pick the one $\neq d_{nn}$)
10 (decimal)	$\{0, 1, \dots, 9\}$	$\{1, 2, 3, 4, 5, 6, 7, 8\}$	works easily — 7+ cho
$b \geq 4$	$\{0, 1, \dots, b-1\}$	$\{1, 2, \dots, b-2\}$	works ($b-3$ choices ≥ 1)

"Safe" digits avoid 0 (would create trailing-0's rep) and b-1 (would create trailing-(b-1)'s rep) — representations of dual-rep reals. Decimal's 8 safe digits make Problem 1 trivial; binary's 0 makes it impossible (without 1

Figure 1.2: Safe digits in base b are those that aren't 0 (so don't trigger trailing-0's representation) and aren't b-1 (so don't trigger trailing-(b-1)'s representation). Decimal has 8 safe digits — plenty. Binary has none and fails.

Ternary has just 1 and needs a multi-digit fix.

The textbook flagged in §8.3 that *binary* diagonalization doesn't directly prove $(0, 1)$ uncountable, for exactly this reason — there are no safe digits in $\{0, 1\}$. Problem 2 of this section deals with the *ternary* (base-3) version, which sits at the awkward boundary: one safe digit isn't enough, so a smarter rule is required (e.g., alternating by parity of the position, or processing two ternary digits as one base-9 digit).

1.9 Remark — How Many Reals Did the Argument Catch?

Strictly, the argument proves the *contrapositive*: any presumed bijection $\mathbb{N} \rightarrow (0,1)$ fails to be surjective — there’s always a missed real r . This *one* missed real is constructed digit-by-digit from the listing.

In fact, the proof catches infinitely many: for each *valid* substitution rule (and there are infinitely many), one constructs a different missed real. Cantor’s later theorem (§8.3 main text) goes further and shows that the cardinality of $(0,1)$ is genuinely *strictly larger* than $\aleph_0$; the present diagonalization is the working proof that $(0,1)$ isn’t countable, which is the first essential step.

1.10 Remark — A Compact Restatement

The argument can be compressed into one paragraph for someone who already knows the framework:

Suppose $(0,1)$ is countable; list it as $r_1, r_2, \dots$ in canonical decimal. Define $r = 0.e_1e_2\dots$ with $e_n = 2$ if $d_{n,n} = 1$ and $e_n = 1$ otherwise. Then $r \in [1/9, 2/9]$ has digits only in $\{1, 2\}$ — hence a unique representation — and differs from each r_n at position n . So r is in $(0,1)$ but not on the list. Contradiction.

The compressed version is just the long version with the four verifications collapsed into “uniqueness + diagonal differs.” All the work is hidden in the choice of substitution rule: it has to keep e_n in $\{1, \dots, 8\}$ so the constructed string represents a *finite-decimal-free* real with a unique decimal expansion. That choice is the whole point of this problem.